

Module 1:

1. Why Performance testing?

- ➔ Performance testing is done to check **how well an application behaves under different load conditions** — not just whether it works, but **how fast, stable, and scalable it is**.
- ➔ Tools to do performance testing are: JMeter, LoadRunner, NeoLoad, etc.

1. ⚡ To check speed (response time)

- How fast does your app respond?
- Example: If a login page takes 10 seconds → users will leave 😊

2. 👤 To handle multiple users (load handling)

- Can your system handle 100 users? 1000 users? 10,000 users?
- Ensures app doesn't crash when traffic increases

3. 📈 To test scalability

- Can your system grow when users increase?
- Example: During a sale (like Flipkart/Amazon), traffic spikes massively

4. 💧 To find breaking point (stress testing)

- At what point does your system fail?
- Helps identify system limits

5. ⌚ To ensure stability (endurance testing)

- Can the system run continuously for hours/days without issues?
- Detects memory leaks, CPU spikes, etc.

6. ⚙️ To identify bottlenecks

- Database slow? Server overloaded? Network delay?
- Helps developers optimize weak areas

7. 💰 To avoid business loss

- Slow apps = frustrated users = lost revenue
- Example: Payment page delay → user cancels order

8. 🎯 To improve user experience

- Fast + smooth app = happy users 😊

📌 Simple Real-Life Example

Think of a **restaurant** 🍽️:

- Functional testing → Can food be served? ✓
- Performance testing → How fast can 50 customers be served at once? 🕒

🔥 Types of Performance Testing

- Load Testing
- Stress Testing
- Spike Testing
- Endurance Testing

2. Types of Performance testing

➔ Following are types of performance testing: Load testing, Stress testing, Spike testing, Endurance testing, Volume testing, Scalability testing.

1. ⚡ Load Testing

🔑 What it means:

Testing system behaviour under **expected number of users**

🔑 Goal:

Check if the system works fine under normal load

🔑 Example:

- 1000 users login at the same time
- System should respond within 2–3 seconds

🔑 Outcome:

- ✓ Response time
- ✓ Throughput
- ✓ No crashes

2. 🔥 Stress Testing

🔑 What it means:

Testing system **beyond its limit**

🔑 Goal:

Find the **breaking point**

☞ **Example:**

- System designed for 1000 users
- You test with 5000 users

☞ **Outcome:**

- ✓ When does it crash?
- ✓ Does it recover after failure?

3. ⚡ Spike Testing

☞ **What it means:**

Sudden increase or decrease in load

☞ **Goal:**

Check how system handles **traffic spikes**

☞ **Example:**

- Users jump from 100 → 10,000 instantly (like flash sale)

☞ **Outcome:**

- ✓ Does system crash?
- ✓ Does it recover quickly?

4. 🕒 Endurance Testing (Soak Testing)

☞ **What it means:**

Testing system for a **long duration**

☞ **Goal:**

Check stability over time

☞ **Example:**

- 500 users continuously for 24 hours

☞ **Outcome:**

- ✓ Memory leaks
- ✓ Performance degradation over time

5. 📈 Scalability Testing

☞ What it means:

Check if system can **scale up or down**

☞ Goal:

Understand how performance changes with load increase

☞ Example:

- Increase users gradually: 100 → 500 → 1000 → 5000

☞ Outcome:

- ✓ Can system handle growth?
- ✓ When do we need more servers?

6. 📊 Volume Testing

☞ What it means:

Testing with **large amount of data**

☞ Goal:

Check database performance

☞ Example:

- Millions of records in DB
- Search/query performance

☞ Outcome:

- ✓ Slow queries
- ✓ DB bottlenecks

3. When do we do performance testing?

➔ You don't do performance testing all the time — you do it at **specific situations where performance matters.**

1. 📦 Before Production Release

☞ Why:

To ensure system can handle real users

☞ **Example:**

- Before launching a CRM or e-commerce app

✓ Avoids crashes after release

✓ Ensures smooth user experience

2. 🧑 When expecting high traffic

☞ **Why:**

To prepare for heavy usage

☞ **Example:**

- Sale events (Flipkart, Amazon)
- Ticket booking systems
- Banking apps on salary day

✓ Prevents downtime

3. 🔄 After major code changes

☞ **Why:**

New code may slow down system

☞ **Example:**

- New feature added
- API changes
- Database changes

✓ Ensures performance is not degraded

4. 🏢 After infrastructure changes

☞ **Why:**

Server/network changes affect performance

☞ **Example:**

- Migrating to cloud
- Changing server configuration

✓ Verifies system stability

5. When system is slow

 Why:

To identify bottlenecks

 Example:

- Login taking too long
- Reports loading slowly

✓ Helps optimize performance

6. For long-running applications

 Why:

To check stability over time

 Example:

- Banking systems
- CRM tools running 24/7

✓ Detects memory leaks

7. When scaling the application

 Why:

To support growth

 Example:

- Users increasing from 1k → 10k

✓ Helps in capacity planning

8. During performance tuning

 Why:

To validate improvements

 Example:

- After optimizing queries
- After caching implementation

✓ Confirms performance gains

➔ Ds

4. What do we test in performance testing

1. ⚡ Response Time

☞ **What:**

Time taken for a request to get a response

☞ **Example:**

- Login should respond within 2–3 seconds

✓ Most important metric

2. 📊 Throughput

☞ **What:**

Number of requests processed per second

☞ **Example:**

- 1000 requests/sec handled by server

✓ Indicates system capacity

3. 👤 Concurrent Users

☞ **What:**

How many users can use the system at the same time

☞ **Example:**

- 500 users login simultaneously

✓ Checks load handling

4. ✖ Error Rate

☞ **What:**

Percentage of failed requests

☞ **Example:**

- 5% requests failing → bad system

✓ Should be minimal

5. 📊 Resource Utilization

☞ **What:**

Usage of system resources

- CPU
- Memory
- Disk
- Network

✓ Helps find bottlenecks

6. 📊 Database Performance

☞ **What:**

How fast DB operations are

☞ **Example:**

- Query execution time
- Slow joins

✓ Critical for backend-heavy apps

7. 🔄 Scalability

☞ **What:**

How system behaves when load increases

☞ **Example:**

- 100 → 1000 → 5000 users

✓ Should scale smoothly

8. 📊 Stability (Endurance)

☞ **What:**

System behavior over long duration

☞ **Example:**

- Run test for 24 hours

✓ Detect memory leaks

5. Performance testing Objective

Key Objectives Explained

1. Response Time 🕒

- Measure how quickly the system responds
👉 Example: Login should respond within 2 seconds
-

2. Throughput 📊

- Number of requests processed per second
👉 Example: 1000 requests/sec
-

3. Stability (Reliability) 🕒

- System should not crash under load
 - No memory leaks or failures
-

4. Scalability 📈

- Ability to handle increasing users
👉 Example: 100 → 1000 users without major degradation
-

5. Identify Bottlenecks 🔍

- Find slow components:
 - Database
 - Server
 - Network
-

6. Resource Utilization 💻

- Check CPU, Memory, Disk usage
👉 Ensure optimal usage
-

7. Validate SLAs (Service Level Agreements) 📄

- Meet performance requirements defined by business
-

8. Concurrency Handling

- How system behaves with multiple users at same time
-

◆ Real-Time Example

 E-commerce website:

- 500 users browsing
- 100 users doing payment

Performance testing ensures:

- Site doesn't crash
 - Payment is fast
 - No errors
-

◆ Tools Used

- Apache JMeter
- BlazeMeter
- LoadRunner

6. sdlc and performance testing life cycle architecture

➔ 1. SDLC (Software Development Life Cycle)

SDLC defines how software is developed step by step.

◆ Phases of SDLC

1. Requirement Gathering

- Understand business needs
- Example: CRM should support 1000 users

2. Design

- System architecture is created
- Database + UI + API design

3. Development

- Developers write code

4. 📄 Testing

- Functional testing
- **Performance testing comes here (mostly)**

5. 🚀 Deployment

- Application released to production

6. 🛠 Maintenance

- Bug fixes, enhancements

✦ Where Performance Testing fits?

☞ Mostly in **Testing phase**

☞ But also:

- During **design (for planning)**
- After deployment (monitoring)

2. Performance Testing Life Cycle (PTLC)

This is the **step-by-step process specifically for performance testing**

🌀 Phases of PTLC

1. 📁 Requirement Analysis

- Understand performance goals
- Example:
 - Response time < 3 sec
 - 1000 concurrent users

2. 📄 Test Planning

- Select tools (JMeter, LoadRunner)
- Define test strategy

3. 📄 Test Design / Scripting

- Create scripts
- Identify scenarios:
 - Login
 - Search
 - Checkout

4. ⚙ Test Environment Setup

- Setup servers similar to production
- Configure tools

5. ► Test Execution

- Run load, stress, spike tests

6. 📊 Result Analysis

- Analyze:
 - Response time
 - Throughput
 - Errors

7. 🔁 Tuning & Re-testing

- Fix issues
- Re-run tests

3. Performance Testing Architecture

This explains **how tools and systems are connected**

Load Generator (JMeter)

↓

Application Server (Web/App Server)

↓

Database Server

1. 🧑 Load Generator

- Tool like JMeter
- Simulates multiple users

2. 🌐 Application Server

- Handles requests
- Business logic execution

3. 🗄 Database Server

- Stores and retrieves data

4. 📊 Monitoring Tools

- Monitor:
 - CPU
 - Memory
 - Network

7. Environments

- a. Development env: 4 Core, 16 RAM
- b. QA Env: 4, 32 RAM
- c. Pre Prod (Performance testing) Env: 8 Core, 124 RAM (Similar to prod env)
- d. Prod Enc: 16 Core, 256 RAM

8. Roles and Responsibility for performance test engineer

1. Requirement Analysis

What you do:

- Understand performance requirements from business/team

Example:

- Response time < 3 sec
- Support 1000 concurrent users

✓ Clarify SLAs (Service Level Agreements)

◆ 2. Test Planning

What you do:

- Decide testing approach and tools

Example:

- Choose **Apache JMeter** or LoadRunner
- Define types of testing (load, stress, spike)

✓ Prepare test strategy document

◆ 3. Script Development

What you do:

- Create performance test scripts

Example:

- Record login, search, checkout APIs
- Parameterization (dynamic data)
- Correlation (handling dynamic values)

✓ Make scripts realistic

◆ 4. Test Environment Setup

☞ **What you do:**

- Prepare environment similar to production

☞ **Example:**

- Configure servers
- Setup test data

✓ Avoid false results

◆ 5. Test Execution

☞ **What you do:**

- Run different types of tests

☞ **Example:**

- Load test → normal users
- Stress test → beyond capacity

✓ Monitor system during execution

◆ 6. Monitoring System Performance

☞ **What you do:**

- Track system metrics

☞ **Example:**

- CPU usage
- Memory usage
- Server logs

✓ Identify bottlenecks

◆ 7. Result Analysis

☞ What you do:

- Analyze test results

☞ Example:

- Response time
- Throughput
- Error rate

✓ Compare with expected SLAs

◆ 8. Bottleneck Identification

☞ What you do:

- Find performance issues

☞ Example:

- Slow database query
- High CPU usage

✓ Work with dev team

◆ 9. Performance Tuning Support

☞ What you do:

- Suggest improvements

☞ Example:

- Optimize queries
 - Add caching
 - Increase server capacity
-

◆ 10. Reporting

☞ What you do:

- Prepare reports for stakeholders

☞ Include:

- Test summary
- Graphs
- Recommendations

✓ Helps decision-making

⚙ 11. Re-testing & Validation

👉 **What you do:**

- Re-run tests after fixes

✓ Ensure issues are resolved

📋 Real-Time Example (CRM Application)

- Analyze → 1000 users requirement
- Script → login, create lead, search
- Execute → simulate users in **Apache JMeter**
- Monitor → CPU, DB
- Report → performance issues
- Interview One-Liner

➔ A performance testing engineer is responsible for analyzing requirements, designing and executing performance tests, monitoring system behavior, identifying bottlenecks, and providing recommendations to ensure the application meets performance and scalability requirements.

Module 2:

9. NFR gathering and strategy

1. NFR Gathering (Non-Functional Requirements)

⚙ What is NFR?

NFR defines **how the system should perform**, not what it does.

👉 Example:

- Functional → User can login
 - NFR → Login should happen within 2 seconds
-

⚡ What do we gather in NFR?

1. ⚡ Response Time

- How fast should the system respond?
 - Example: Login < 2 sec
-

2. 👤 Concurrent Users

- How many users at the same time?
 - Example: 1000 users
-

3. 📊 Throughput

- Requests per second
 - Example: 500 transactions/sec
-

4. ✖ Error Rate

- Allowed failures
 - Example: < 1% errors
-

5. 📈 Peak Load

- Maximum expected traffic
 - Example: Sale time → 5000 users
-

6. ⌚ Test Duration

- How long system should run
 - Example: 8 hours (endurance test)
-

7. 🌐 Environment Details

- Servers, DB, network setup
-

8. 🎯 SLA (Service Level Agreement)

- Performance targets agreed with client
-

✦ From where do we gather NFR?

- Business team
 - Product owners
 - Architects
 - Past production data
 - Monitoring tools
-

📄 Example (CRM Application)

- 1000 concurrent users
 - Response time < 3 sec
 - Error rate < 2%
-

🚀 2. Performance Testing Strategy

Strategy = **How you plan to test performance**

✦ What should be included in Strategy?

1. 🎯 Scope of Testing

- What modules to test?
 - Example:
 - Login
 - Search
 - Reports
-

2. 📄 Types of Testing

- Load testing
 - Stress testing
 - Spike testing
 - Endurance testing
-

3. 🛠 Tools Selection

- Example:
 - **Apache JMeter**
 - LoadRunner
-

4. 🖥 Test Environment

- Should match production
-

5. Workload Modeling

 How users behave:

- 60% login
 - 30% search
 - 10% reports
-

6. Test Scenarios

Example:

- 100 users → 500 → 1000 gradually
 - Sudden spike to 5000 users
-

7. Entry & Exit Criteria

 Entry:

- Application stable
- Environment ready

 Exit:

- SLA met
 - No critical issues
-

8. Monitoring Plan

- CPU
 - Memory
 - DB performance
-

9. Reporting Plan

- What reports to generate
 - Metrics to include
-

📝 Simple Difference

NFR Gathering 📁

What to achieve

Requirements

Example: 1000 users

Strategy 🎯

How to achieve

Plan

How to simulate 1000 users

Interview Answer (Short & Perfect)

NFR gathering involves collecting performance requirements like response time, concurrent users, throughput, and SLAs from stakeholders. Based on these, we design a performance testing strategy that defines scope, test types, workload modeling, tools like Apache JMeter, environment setup, execution plan, and reporting to ensure the system meets performance expectations.

Module 3:

10.JMeter setup

🔧 1. Prerequisites

Before installing JMeter, you need:

✓ Java (Mandatory)

- Install **Java Development Kit** (JDK 8 or higher recommended)
- Check installation:

`java -version`

🔗 If not installed, download from **Oracle Corporation** or use OpenJDK.

📥 2. Download JMeter

- Go to official site:
🔗 **Apache JMeter** (Binary)
- Download the **ZIP (Windows)** or **TGZ (Mac/Linux)** file

📂 3. Extract Files

- Extract the downloaded file to any folder
Example:

C:\JMeter

⚙️ 4. Set Environment Variables (Optional but Recommended)

▶ Add JAVA_HOME

1. Go to Environment Variables
2. Add:

JAVA_HOME = C:\Program Files\Java\jdk-XX

▶ Add JMeter to PATH

Add:

C:\JMeter\bin

▶ 5. Launch JMeter

On Windows:

- Go to:

bin → jmeter.bat

- Double click OR run:

jmeter

On Mac/Linux:

./jmeter

🔧 6. Verify Installation

- JMeter GUI should open
 - You'll see:
 - Test Plan
 - Thread Group
 - Samplers
-

⚡ 7. Quick Test (Optional)

1. Right-click **Test Plan**
 2. Add → Threads → Thread Group
 3. Add → Sampler → HTTP Request
 4. Add → Listener → View Results Tree
 5. Click ▶ Run
-

📖 Pro Tips

- Use GUI only for **test creation**
- Run tests in CLI for better performance:

jmeter -n -t test.jmx -l result.jtl

11. JMeter Installation

➔ What to Download?

You should always download the **Binary version**, NOT the source.

- ✓ **Binary** → Ready to run
- ✗ **Source** → Needs compilation (not needed for normal use)

Download from:

🔗 [Apache JMeter official website](#)

🔗 Prerequisite

JMeter requires Java.

1. Install Java

- Install **JDK 8 or higher**
- Verify installation:

`java -version`

If not installed, download:

🔗 [Java Development Kit](#)

🔗 Installation Steps (Windows)

2. Download JMeter Binary

- Download .zip file (e.g., apache-jmeter-5.x.zip)

3. Extract the Zip

- Extract to any folder
Example: C:\jmeter

4. Set Environment Variables (Optional but Recommended)

Add JAVA_HOME

- Variable Name: JAVA_HOME
- Value: Path to JDK
Example:

C:\Program Files\Java\jdk-17

Add JMeter to PATH

- Add:

C:\jmeter\bin

🔑 Run JMeter

Go to:

C:\jmeter\bin

Run:

- jmeter.bat (Windows)
- jmeter.sh (Mac/Linux)

👉 GUI will launch 🖱️

🔑 Verify Installation

Once opened:

- You should see JMeter UI
 - No Java errors → Setup is correct
-

🔑 Folder Structure (Important)

Inside JMeter:

- bin → Run files (important)
 - lib → libraries
 - extras → plugins/tools
-

🔑 Common Issues & Fixes

❌ 'java not recognized'

→ Java not installed or PATH not set

❌ JMeter not opening

→ Check Java version (use JDK, not just JRE)

🔑 Pro Tips

- Use **non-GUI mode** for performance testing:
jmeter -n -t test.jmx -l result.jtl

- Install plugins via:
👉 JMeter Plugins Manager

- JMeter path : C:\Users\User\Downloads\apache-jmeter-5.6.3\apache-jmeter-5.6.3\bin

12. Class path setup for jmeter recording

◆ 1. Basic Understanding

In Apache JMeter:

- You **don't manually set CLASSPATH for recording**
- JMeter automatically loads required classes from:
 - lib/
 - lib/ext/

☞ You only add classpath **when using external dependencies (like JDBC, custom jars, etc.)**

◆ 2. For HTTP(S) Recording (Most Common Case)

✓ **No Classpath Needed**

If you are using:

- **HTTP(S) Test Script Recorder**

Then:

- No classpath setup required ✕
 - Just configure proxy & certificate
-

◆ Steps for Recording

1. Add Recorder

Test Plan → Add → Non-Test Elements → HTTP(S) Test Script Recorder

2. Set Port (e.g., 8888)

3. Start Recorder

◆ 3. Install JMeter Root Certificate (VERY IMPORTANT)

Without this, HTTPS recording won't work.

Steps:

1. Start recorder once → JMeter creates:

bin/ApacheJMeterTemporaryRootCA.crt

2. Install certificate in browser:

- Chrome → Settings → Security → Manage Certificates → Chrome Root Store → Export.
- Import .crt file under **Trusted Root**

🔑 4. Browser Proxy Setup

Set proxy in browser:

Host: localhost

Port: 8888

🔑 5. When DO You Need Classpath?

You need it only in these cases:

✓ JDBC Testing

Add DB driver:

lib/

Example:

- MySQL → mysql-connector.jar
-

✓ Custom Java Code / Samplers

Add .jar to:

lib/ext/

✓ SOAP / API with external libs

Same:

lib/

🔑 6. Advanced: Setting Classpath Manually

If needed, edit:

jmeter.bat or jmeter.sh

OR use:

set CLASSPATH=yourjar.jar

☞ But this is rarely required.

◆ 7. Common Issues in Recording

✗ HTTPS not capturing

→ Certificate not installed

✗ Blank response

→ Proxy not set in browser

✗ Some requests missing

→ Add include/exclude patterns in recorder

◆ 8. Pro Tip (Important for Interviews 💡)

☞ “Classpath is not required for JMeter recording. It is only required when integrating external dependencies like JDBC drivers or custom samplers.”

13. Increase System TCP/IP ports

◆ Steps to Increase TCP/IP Ports (Windows)

✓ 1. Check Current Range

netsh int ipv4 show dynamicport tcp

✓ 2. Increase Port Range

netsh int ipv4 set dynamicport tcp start=10000 num=55535

✓ Expands available ports significantly

✓ 3. Restart System

☞ Required for changes

14. Increase Jmeter heap memory

Increasing **heap memory in Apache JMeter** is very important when running large tests — otherwise you'll hit:

- OutOfMemoryError
 - JMeter crashes / freezes
 - Slow execution
-

◆ What is Heap Memory?

- Memory used by JMeter (Java process)
 - Controlled using **JVM arguments (-Xms, -Xmx)**
-

◆ Step-by-Step: Increase Heap Memory (Windows)

✓ Step 1: Go to JMeter bin folder

Example:
apache-jmeter/bin

✓ Step 2: Open File
jmeter.bat

✓ Step 3: Find This Line
set HEAP=-Xms1g -Xmx1g -XX:MaxMetaspaceSize=256m

✓ Step 4: Increase Values

Example:
set HEAP=-Xms2g -Xmx4g -XX:MaxMetaspaceSize=512m

📖 Meaning:

- -Xms2g → Initial memory = 2GB
 - -Xmx4g → Max memory = 4GB
-

◆ For macOS / Linux

Open:
jmeter.sh

Modify:
HEAP="-Xms2g -Xmx4g -XX:MaxMetaspaceSize=512m"

⚙ Recommended Heap Size

RAM Available JMeter Heap

4 GB	1–2 GB
8 GB	2–4 GB
16 GB	4–8 GB

📌 Don't use 100% RAM — leave memory for OS

⚙ Verify Heap Settings

Run JMeter → Check logs:
jmeter.log

Or use:
jmeter -v

⚙ Best Practices 💡

✓ Use Non-GUI Mode

jmeter -n -t test.jmx -l result.jtl

✓ Avoid Heavy Listeners

✗ View Results Tree (in load test)

✓ Use Summary Report / Backend Listener

✓ Use CSV instead of hardcoded data

✓ Disable unnecessary elements

⚙ Common Mistakes ✗

- Setting heap > system RAM → crash
 - Running large test in GUI mode
 - Not increasing metaspace
-

⚙ Real Example 💡

If you run:

10,000 users with 1GB heap ✗

→ JMeter crashes

But:

10,000 users with 4GB heap + non-GUI ✓

→ Smooth execution

Interview Answer

We increase JMeter eap memory by modifying -Xms and -Xmx values in jmeter.bat or jmeter.sh based on system RAM, and always run large tests in non-GUI mode to avoid memory issues.

Module 4:

15.Proxy setup

◆ Why Proxy Setup is Needed?

- Browser → sends request to **JMeter Proxy**
- JMeter → forwards to server & records requests

☞ Without proxy → JMeter **cannot capture traffic**

◆ Step 1: Configure JMeter Recorder

1. Open JMeter
2. Add:

Test Plan → Add → Non-Test Elements → HTTP(S) Test Script Recorder

3. Set Port:8888 (or any free port)
4. Click **Start**

◆ Step 2: Setup Proxy in Browser

◆ For Google Chrome (Windows)

1. Open Chrome
2. Go to:

Settings → System → Open your computer's proxy settings

◆ In Windows Proxy Settings

1. Enable:
Manual Proxy Setup = ON

2. Enter:
Address: localhost
Port: 8888

3. Save

◆ Alternative (Direct Shortcut)

- Press:
Windows + R → inetctl.cpl
 - Go to:
Connections → LAN Settings
 - Enable:
Use a proxy server
 - Set:
Address: localhost
Port: 8888
-

◆ Step 3: Install JMeter Certificate (HTTPS)

1. Start recorder once
2. Go to JMeter bin folder
3. Find:
ApacheJMeterTemporaryRootCA.crt
4. Import in Chrome:
 - Settings → Privacy & Security → Security
 - Manage Certificates
 - Import into **Trusted Root**

16. Jmeter Recording

- ➔ Test Plan: Test plan consist of the actions and the components we need to execute the performance test script.
- ➔ In Jmeter we can work only on one project at a time, if we want to work on another project

✓ Step 1: Open JMeter

- Run:

jmeter.bat

✓ Step 2: Add Thread Group

Test Plan → Add → Threads → Thread Group

✓ Step 3: Add HTTP(S) Test Script Recorder

Test Plan → Add → Non-Test Elements → HTTP(S) Test Script Recorder

✓ Step 4: Configure Recorder

Set:

Port: 8888

Optional:

- Add **Include/Exclude patterns** (to filter requests)
-

✓ Step 5: Add HTTP Request Defaults (Recommended)

Test Plan → Add → Config Element → HTTP Request Defaults

Set:

- Server Name → your application URL
-

✓ Step 6: Add Recording Controller

Thread Group → Add → Logic Controller → Recording Controller

📁 All recorded requests will be stored here

✓ Step 7: Start Recorder

- Click **Start** in recorder
-

✓ Step 8: Setup Browser Proxy

In your browser (like Google Chrome):

- Proxy:

Host: localhost

Port: 8888

✓ Step 9: Install JMeter Certificate (HTTPS)

1. Go to JMeter bin folder
2. Find:

ApacheJMeterTemporaryRootCA.crt

3. Import into browser:

- Trusted Root Certificate
-

✓ Step 10: Perform Actions

- Open browser
 - Navigate app (login, search, etc.)
 - JMeter captures all requests
-

✓ Step 11: Stop Recording

- Click **Stop** in recorder

17.Jmeter script recording with other tools (Blazemeter, tar to JMX)

Method 1: Using BlazeMeter Chrome Extension (Easiest)

Step 1: Install Extension

- Install **BlazeMeter Chrome Extension** from Chrome Web Store
-

Step 2: Start Recording

1. Click on BlazeMeter extension icon
 2. Click **“Start Recording”**
 3. Choose:
 - Record new test
 - Protocol: HTTP/HTTPS
-

Step 3: Configure (Important)

- Give test name
 - Select **JMeter** as script type
 - Enable:
 - Capture headers
 - Capture cookies
-

Step 4: Record Your Flow

- Open your application (e.g., login page)
- Perform actions:
 - Login

- Search
 - Navigate pages
 - BlazeMeter captures all requests automatically
-

Step 5: Stop Recording

- Click **Stop**
 - Script gets saved in BlazeMeter
-

Step 6: Export to JMeter

- Click **Download JMX**
 - Open it in **Apache JMeter**
-

⚡ Method 2: Using JMeter Recorder + BlazeMeter

If you want more control:

Step 1: Setup JMeter Recorder

- Open JMeter
 - Add:
 - Thread Group
 - HTTP(S) Test Script Recorder
-

Step 2: Configure Proxy

- Default port: **8888**
 - Set browser proxy to:
 - Host: localhost
 - Port: 8888
-

Step 3: Install Certificate

- Install JMeter certificate in browser (for HTTPS recording)
-

Step 4: Record Steps

- Start recorder in JMeter
- Perform application steps in browser

Step 5: Upload to BlazeMeter

- Save script (.jmx)
- Upload it to BlazeMeter
- Run test in cloud

18. Playback and validation of request and response

➔ Playback + validation in **Apache JMeter** is a core step after recording (via HAR or BlazeMeter). The goal is to make sure your script behaves exactly like a real user.

1. What is Playback in JMeter?

- ➔ Playback = Running your recorded script
- ➔ Executes HTTP requests in sequence
 - Sends them to the server
 - Receives responses
 - How to Do Playback:
 - Steps:
 - Open your .jmx script
 - Add Listener:
 - **View Results Tree**
 - Click ► **Run**

2. Validate Request

- ➔ To perform request validation in we can use **Debug Sampler**.
- ➔ We don't have any direct assertions to do request validation, so we can use:
 - JSR223 Assertions
 1. JSR223
- ➔ JSR223 = Scripting using standard Java APIs
- ➔ Uses languages like:
 - **Groovy (most recommended)**
 - JavaScript (less used)
- ➔ In JMeter, **Groovy + JSR223 = Best practice**
- ➔ Right click on the HTTP Request -> Go to Assertion -> Select JSR223 Assertion -> Write the groovy code
- ➔ Eg:

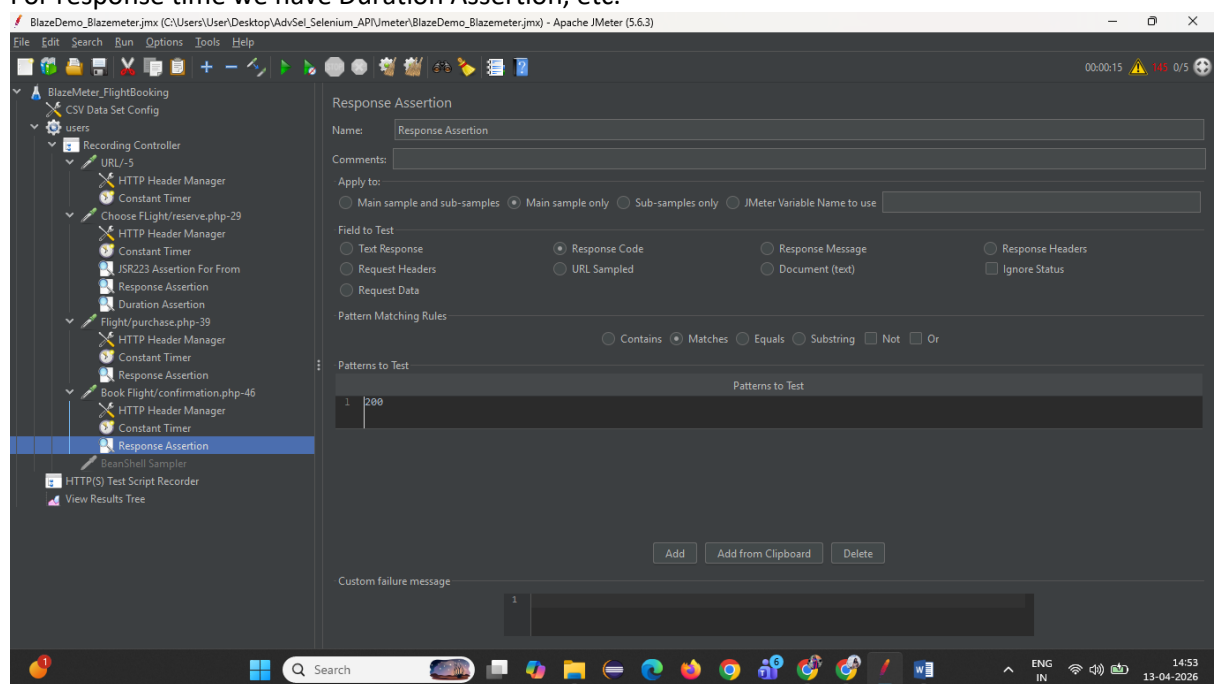
```
def loc=prev.getSamplerData()
if(!loc.contains("fromPort- ${From}")) {
    AssertionResult.setFailure(true)
    AssertionFailure.setFailureMessage( "From is not selected correctly")
}
```

- ➔ Here
- ➔ Line 1: prev means previous sampler result (request + response info)
- ➔ Gets the **actual request sent by JMeter**
- ➔ Stores it in variable loc

- ➔ Example value of loc:
 - POST http://blazedemo.com
 - fromPort=Paris&toPort=London
- ➔ getSamplerData() Get the complete request data that was sent by the sampler.
- ➔ Line 2: It checks does the request contains fromPort=\${From}
- ➔ \${From} is Jmeter variable
- ➔ ! operator, if condition is **FALSE** (value NOT found)
- ➔ Then execute failure block
- ➔ Line 3 & 4: to make the assertion as pass which means the execution failed and the msg will be printed with error.

3. Validate Response:

- ➔ To do response assertion we have Response Assertion where we can do assertion for status code, status message, Response header.
- ➔ For response time we have Duration Assertion, etc.



- ➔ We can even JSR223 Assertions as well

Extra Required Knowledge with respect to regex

Regular Expressions (Regex)

1. Basic Building Blocks

🔧 . (dot)

🔑 Matches **any single character**

Example:

a.b

✓ Matches: acb, a1b, a@b

🔧 \d

🔑 Matches **digits (0–9)**

\d

✓ 5, 9

🔧 \w

🔑 Matches **word characters**

✓ Letters + numbers + underscore

\w

✓ A, z, 5, _

🔧 \s

🔑 Matches **whitespace**

✓ Space, tab

🔥 2. Quantifiers (How many times)

🔧 *

🔑 0 or more times

ab*

✓ a, ab, abb, abbbb

🔧 +

🔑 1 or more times

ab+

✓ ab, abb

✗ Not a

◆ ?

☞ 0 or 1 time (optional)
colou?r

✓ color, colour

◆ {n}

☞ Exactly n times
\d{3}

✓ 123

◆ {n,m}

☞ Between n and m times
\d{2,4}

✓ 12, 123, 1234

🔥 3. Character Classes

◆ [abc]

☞ Match any one character

✓ a, b, or c

◆ [A-Z]

☞ Uppercase letters

◆ [0-9]

☞ Numbers

◆ [^abc]

☞ NOT a, b, or c

🔥 4. Capturing Groups (VERY IMPORTANT in JMeter)

🔗 ()

🔗 Used to **extract values**

Example:

id=(\d+)

✓ Extracts:

12345

🔗 Used in JMeter as:

\$1\$

🔥 5. Common Patterns Used in JMeter

✓ Extract Number

(\d+)

✓ 12345

✓ Extract Text Between Quotes

"(.+?)"

✓ hello

✓ Extract Token

token": "(.+?)"

✓ Extract Session ID

sessionId=(\w+)

✓ Extract Flight Number

([A-Z]{2}\d+)

✓ UA954

🔥 6. Greedy vs Non-Greedy

✗ Greedy

(.+)

🔗 Takes too much

✓ Non-Greedy
(.+)?

☞ Takes exact value

✓ Always use this in JMeter

🔥 7. Anchors (Position-based)

☞ ^

☞ Start of line
^Hello

☞ \$

☞ End of line
world\$

🔥 8. Real-Time Example (JMeter)

Response:

<input value="ABC123">

Regex:

value="(.+?)"

Extracted:

ABC123

☞ Easy Way to Remember

- \d → digits
 - \w → word
 - \s → space
 - + → many
 - * → optional many
 - () → capture
-

☞ Interview Answer (Short)

☞ "Regular expressions are patterns used to extract dynamic values. Common components include character classes, quantifiers, capturing groups, and special symbols like \d, \w, and .*"

Pro Tip

 In **JMeter**, you'll mostly use:

- `(.+?)`
- `(\d+)`
- `(\w+)`
- `[A-Z]{2}\d+`